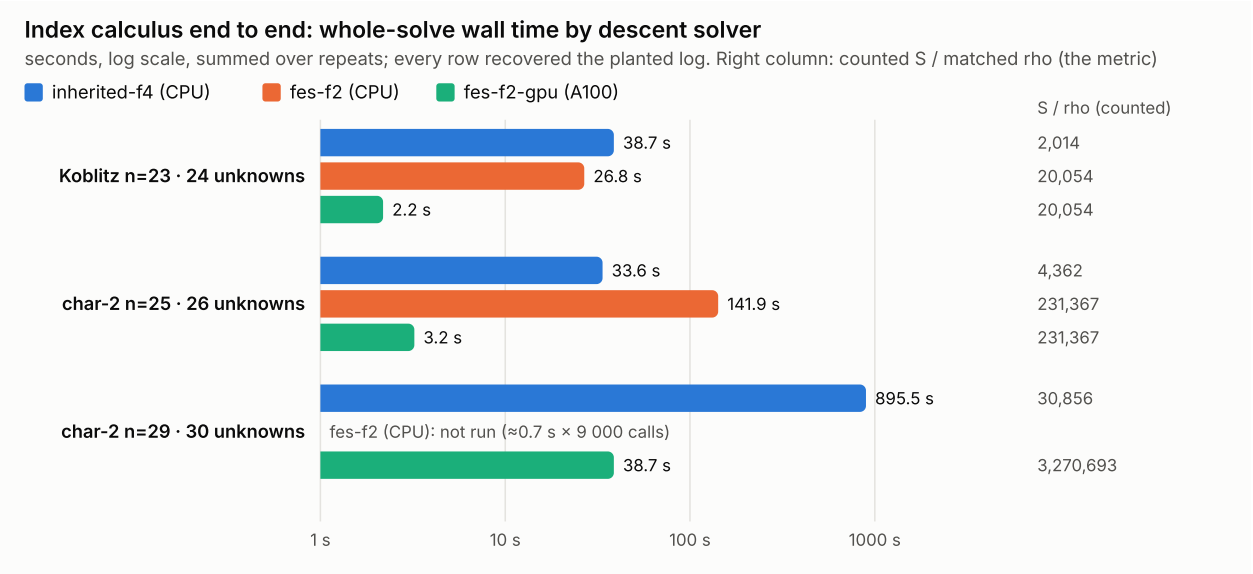
fes-f2-gpu: the FES GPU worker in a real index-calculus relation phase

User request (2026-11-13): hook the FES GPU worker into a real relation phase for the first end-to-end index-calculus number. Until now only the `icx fes` demo called `gpu/fes`.

Standing. All numbers `[pcc:unverified]`: one pod, measured by the session that wrote the code, no validator. **The metric is counted cost S; wall time is a practicality note** (`AGENTS.md` §6). `fes-f2-gpu` runs the same search as `fes-f2` and reports the same counted cost by construction, so it cannot move S: it is an engineering change to wall time, not an advance (§3 class: engineering). No scoreboard or leaderboard row.



End-to-end wall time and counted S, [pcc:unverified]

What was built (commits `0d28a6e605`, `abecd26901`)

- **`gpu/fes --serve`** (CUDA worker `fes.cu`, host worker `fes_solve.cpp`, shared protocol in `fes_io.hpp`): one process, one CUDA context and one set of device buffers for a whole run; systems stream in on stdin, replies are `SOLUTIONS <listed> <total>` plus the points. `total > listed` flags a capped list.
- **`fes_gpu.rs`**: `ServeWorker` and a process-wide shared server (started on first use from `ICX_FES_WORKER`, restarted after any error).
- **Engine `fes-f2-gpu`** (`ic_framework/solvers.rs`): `FesF2`'s body is now `fes_quadratic_solve(system, search)`, used by both engines, so form building, the `word XORs (Gray-code steps)` count, `bitop/v1` count and CPU verification of every point are shared [pcc:unverified]. The GPU engine returns `Unsatisfiable` only on a complete list, falls back to the CPU search (flagged `gpu_fallback`) without a worker, and accepts up to 40 unknowns.
- **Order.** The descent oracle keeps the *first* solution that lifts. The CUDA kernel appends zeros in arbitrary thread order, so the first run (kept as `raw/run1-unordered-koblitz-n23.json`) followed a different trajectory from `fes-f2` (2642 against 1878 calls, both verified). Proposals are now sorted by Gray rank, the order of `fes-f2`'s sequential walk; after that every `fes-f2-gpu` row equals its `fes-f2` row in calls, hit rate and S.

- **Tests**: serve protocol against the host worker on 40 systems; shuffled lists sort back to `fes-f2`'s order on 60 systems; `fes-f2-gpu` counts like `fes-f2`; the every-engine agreement tests (random and descent systems) include it.

Run

`ic bench --sweep` (full DLP: factor base, relation phase with the descent oracle, linear algebra, planted-log check, matched counted rho with 16 runs), solver budget 120 s per call. Pod `02rw1tw9o5v1hm`, A100 80GB PCIe (Runpod host `3is1tqstotnl`), driver 595.91.07, CUDA 12.8; `ic` release binary built from `abecd26901` (sha256 `ad9084a9…`), shipped to the pod; `fes_cuda` built there (compiled-in selftest MATCH; serve-protocol smoke test byte-identical to the host worker). Sweeps in `sweeps/`; raw output `raw/out_e2e.tgz`; tables `RESULTS.md`. 03:56Z–04:20Z, about $1.80 with the first run.

Results

Counted S / matched rho is the metric; wall columns are the practicality note.

instance (unknowns)	solver	verified	hit rate	solver calls	S / rho	solver wall	whole solve
Koblitz n=23 (24), 2 repeats	`inherited-f4`	y	0.6238	2487	**2,014**	38.5 s	38.7 s
	`fes-f2`	y	0.6225	1878	20,054	26.6 s	26.8 s
	`fes-f2-gpu`	y	0.6225	1878	20,054	**1.88 s**	**2.18 s**
char-2 n=25 (26), 1 repeat	`inherited-f4`	y	0.6396	1193	**4,362**	33.5 s	33.6 s
	`fes-f2`	y	0.6369	2338	231,367	141.5 s	141.9 s
	`fes-f2-gpu`	y	0.6369	2338	231,367	**2.77 s**	**3.22 s**
char-2 n=29 (30), 1 repeat	`inherited-f4`	y	0.6305	8977	**30,856**	893 s	895.5 s
	`fes-f2-gpu`	y	0.6305	8977	3,270,693	**32.5 s**	**38.7 s**

Derived wall ratios (practicality note):

- `fes-f2` → `fes-f2-gpu`, same trajectory and S: whole solve **12.3×** faster at 24 unknowns, **44.1×** at 26 [pcc:unverified].
- `inherited-f4` → `fes-f2-gpu`: whole solve **17.7×**, **10.4×** and **23.1×** faster at 24, 26 and 30 unknowns, while `inherited-f4`'s counted S stays 10×, 53× and 106× lower [pcc:unverified] (9.96× at 24 unknowns; the first version rounded it to 9×, corrected after `research/ic_gpu_replay_20261114/VALIDATION_REVIEW.md`).
- `fes-f2` at n=29 was not run: about 0.7 s per call at 30 unknowns (`research/inherited_f4_next_20261112`) over 8977 calls is roughly 1.7 hours, and its S equals `fes-f2-gpu`'s by construction.

Reading

- The relation phase now runs on the GPU end to end and reproduces the CPU engine exactly; this is the first end-to-end IC number with the `gpu/fes` kernel in the loop [pcc:unverified].
- It does **not** lower the IC baseline in the repository's metric [pcc:unverified]: S is unchanged against `fes-f2`, and `inherited-f4` remains the cheaper engine in counted cost at every size here. The gap to matched rho is unchanged (best row 2,014× at n=23, 4,362× at n=25, 30,856× at n=29, all `inherited-f4`).
- Per call, `fes-f2-gpu` costs 1.0, 1.2 and 3.6 ms at 24, 26 and 30 unknowns against kernel times of roughly 0.1, 0.2 and 1.3 ms (A100 kernel rounds); the rest is the text pipe, form packing and CPU verification, the next thing to cut.

Not done / next

- Binary serve protocol and batching several systems per launch (the per-call overhead above).
- A size rule (`fes-f2-gpu` above some n, `inherited-f4` below) is a wall-time choice only; in counted units `inherited-f4` wins, so no such rule is proposed as an improvement.
- An independent replay of these rows before they are cited as more than `[pcc:unverified]`.